UNIVERSIDADE ESTADUAL DA PARAÍBA

CENTRO DE CIÊNCIAS E TECNOLOGIA - CCT

DEPARTAMENTO DE COMPUTAÇÃO - DC

JOHNATAN RODRIGUES DOS SANTOS

LÍVIA NASCIMENTO OLIVEIRA

PROBLEMA DA PARTIÇÃO DE CONJUNTOS

CAMPINA GRANDE - PB

2026

SUMÁRIO

SUMÁRIO 2
1. INTRODUÇÃO 3
1.1 Contextualização do problema 3
2. METODOLOGIA E IMPLEMENTAÇÃO 4
2.1 Arquitetura da Solução e Ferramentas 4
2.2 Pré-processamento e Estruturas de Dados 5
2.3 Modelagem de Passagem e Estado das Instâncias 6
3. ALGORITMOS IMPLEMENTADOS 7
3.1 Backtracking 7
3.1.1. Análise de Corretude 7
3.1.2. Análise de Desempenho Computacional 8
3.1.2.1. Complexidade Espacial 8
3.1.2.2. Temporal 8
3.2. Branch and Bound 8
3.2.1. Análise de Corretude 9
3.2.2. Análise de Desempenho Computacional 10
3.2.2.1. Complexidade Espacial 10
3.2.2.2. Complexidade Temporal 10
3.3. Programação Dinâmica 10
3.3.1. Análise de Corretude 10
3.3.2. Análise de Desempenho Computacional 11
3.3.2.1. Complexidade Espacial 11
3.3.2.2. Complexidade Temporal 11
3.4. Estratégia Gulosa 11
3.4.1. Análise de Corretude 12
3.4.2. Análise de Desempenho Computacional 13
3.4.2.1. Complexidade Espacial 13
3.4.2.2. Complexidade Temporal 13
4 EXPERIMENTOS E RESULTADOS 13
4.1 Instâncias de Teste 13
4.2 Resultados Obtidos 14
5. ANÁLISE E DISCUSSÃO 15
5.1 Comparação das Estratégias: Exatidão vs. Heurística 15
6. CONCLUSÃO 17
7. REFERÊNCIAS 18

1. INTRODUÇÃO

1.1 Contextualização do problema

O Problema da Partição de Conjuntos (*Set Partition Problem*) é um dos problemas clássicos da teoria da complexidade computacional e da otimização combinatória. Ele consiste em um problema de decisão definido da seguinte forma: dado um multiconjunto $S = \{a_1, a_2, \ldots, a_n\}$ composto por inteiros estritamente positivos, deseja-se determinar se é possível particioná-lo em dois subconjuntos disjuntos, S_1 e S_2, de modo que a soma dos elementos de ambos seja exatamente igual.

Formalmente, o problema pode ser descrito pelas seguintes condições:

$S1 \cup S2 = S$, isto é, a união dos subconjuntos corresponde ao conjunto original;

$S1 \cap S2 = \emptyset$, ou seja, os subconjuntos são mutuamente exclusivos;

$\sum_{x \in S1} x = \sum_{y \in S2} y$, garantindo a igualdade entre as somas dos elementos.

Esse problema pode ser visto como um caso particular do Problema da Soma de Subconjuntos (*Subset Sum Problem*). Historicamente, ele ganhou destaque ao integrar o conjunto dos 21 problemas clássicos apresentados por Richard Karp em 1972, nos quais foi demonstrado que pertencem à classe dos problemas NP-completos (KARP, 1972). Pertencer à classe NP-completo implica que, embora seja possível verificar a validade de uma solução em tempo polinomial, não há, até o momento, conhecimento de um algoritmo determinístico que resolva o problema de forma geral também em tempo polinomial. Isso torna sua resolução computacionalmente desafiadora, especialmente para instâncias de grande porte. Diante dessa complexidade, abordagens ingênuas baseadas em força bruta, que avaliam todas as 2^n possíveis combinações, tornam-se rapidamente inviáveis. Assim, torna-se fundamental o uso de técnicas mais sofisticadas que reduzam o espaço de busca, como estratégias de poda e limitação. Nesse contexto, o presente

trabalho tem como objetivo explorar e analisar implementações deste problema sob quatro perspectivas algorítmicas: Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa.

2. METODOLOGIA E IMPLEMENTAÇÃO

2.1 Arquitetura da Solução e Ferramentas

O desenvolvimento algorítmico do presente projeto foi realizado na linguagem de programação Java, com o objetivo de alcançar elevado desempenho por meio do paradigma de orientação a objetos e da tipagem forte. A arquitetura da solução foi estruturada com base no princípio da separação de responsabilidades, promovendo maior coesão e organização entre os componentes do sistema.

A estrutura do sistema foi dividida nos seguintes módulos:

- **Manipulação de entradas e saídas (*FileHandler* e *Main*):** responsável pela leitura de dados a partir de arquivos no formato .txt, bem como pela formatação da saída contendo as partições resultantes, conforme o modelo estabelecido;
- **Modelagem de resposta (*PartitionResult*):** consiste em um objeto imutável que encapsula o resultado da execução, contendo um valor booleano indicativo de viabilidade e, quando aplicável, as duas partições resultantes;
- **Camada de resolução lógica (BacktrackingSolver, BranchAndBoundSolver, DynamicProgrammingSolver e GreedySolver):** Composta pelas classes responsáveis pela implementação das diferentes estratégias de solução do problema. As abordagens de Backtracking e Branch and Bound baseiam-se na exploração sistemática do espaço de busca, com aplicação de podas no segundo caso. A abordagem por Programação Dinâmica utiliza estruturas de armazenamento intermediário para evitar recomputação, enquanto a Estratégia Gulosa fundamenta-se em decisões locais sequenciais para construção de uma solução eficiente;
- **Orquestrador de resolução (PartitionSolver):** atua como uma fachada (*Facade*) estrutural, sendo responsável por centralizar a recepção do problema, realizar os pré-processamentos necessários, como verificação de

paridade e cálculo de limites, e delegar a execução para o algoritmo apropriado, conforme a abordagem selecionada.

2.2 Pré-processamento e Estruturas de Dados

Para ambas as abordagens adotadas, a estratégia de força bruta é substituída por mecanismos de busca sistemática com aplicação de técnicas de poda (pruning). Com o intuito de aumentar a eficiência dessas podas, foi implementada uma etapa de pré-processamento dos dados inicialmente, considera-se K como a soma total dos elementos do multiconjunto S, conforme a expressão a seguir:

$$K = \sum_{i=1}^{n} a_i$$

A primeira verificação baseia-se no princípio da paridade. Como o problema exige a divisão do conjunto em dois subconjuntos com somas iguais, é necessário que o valor total K seja par. Caso $K \bmod 2 \neq 0$, conclui-se imediatamente pela impossibilidade de solução, evitando a execução de etapas computacionalmente custosas. Quando a condição de paridade é satisfeita, define-se um valor-alvo (*targetSum*) igual a $\frac{K}{2}$. Essa estratégia reduz o problema a uma única condição de verificação: encontrar um subconjunto S_1 cuja soma seja exatamente igual ao valor-alvo. Uma vez atingido esse valor, o subconjunto complementar S_2 automaticamente satisfaz as condições do problema.

A partir dessa modelagem, são empregadas estruturas auxiliares específicas para cada estratégia, com o objetivo de otimizar o desempenho computacional. No caso do Branch and Bound, utiliza-se um vetor de somas sufixas, que permite estimar limites superiores ao longo da exploração do espaço de busca. Essa estrutura possibilita a realização de podas antecipadas, interrompendo a exploração de ramos que não podem conduzir a uma solução válida. Na abordagem de Programação Dinâmica, emprega-se uma matriz booleana bidimensional de dimensões $(n + 1) \cdot (targetSum + 1)$, na qual cada posição indica se uma determinada soma pode ser obtida a partir de um subconjunto dos primeiros *i*

elementos. Essa estrutura elimina recomputações de subproblemas, caracterizando uma abordagem de complexidade pseudo-polinomial O(n · *targetSum*). Por sua vez, a Estratégia Gulosa baseia-se na ordenação prévia dos elementos em ordem decrescente, de modo que os maiores valores sejam considerados prioritariamente durante a construção dos subconjuntos. Essa heurística visa reduzir a discrepância entre as somas parciais, contribuindo para a obtenção de soluções de boa qualidade em tempo reduzido.

2.3 Modelagem de Passagem e Estado das Instâncias

No que se refere ao gerenciamento de memória, optou-se pela utilização de estruturas do tipo *ArrayList*, passadas por referência ao longo das chamadas recursivas. Essa abordagem resulta em um consumo espacial de ordem O(n), sendo eficiente para o contexto do problema. Os subconjuntos em construção, representados por *currentSubset1* e *currentSubset2*, comportam-se como uma pilha, na qual elementos são inseridos durante a exploração de um ramo da árvore de decisão e removidos no processo de retorno da recursão (*backtracking*), permitindo a exploração de novas possibilidades. Essa estratégia de manipulação por referência é fundamental para evitar a realização de cópias profundas (*deep copy*), que poderiam impactar negativamente o desempenho, especialmente considerando a complexidade exponencial $O(2^n)$ do problema. A cópia das estruturas de dados é realizada apenas no momento em que uma solução válida é encontrada, garantindo maior eficiência no uso de memória. As abordagens baseadas em Backtracking e Branch and Bound utilizam esse modelo recursivo de exploração, no qual o estado da computação é mantido implicitamente pela pilha de chamadas. Por outro lado, a Programação Dinâmica adota uma estratégia iterativa (*bottom-up*), na qual o estado é explicitamente armazenado em uma matriz de memorização, eliminando recomputações de subproblemas e dispensando o uso de recursividade. Já a Estratégia Gulosa baseia-se em uma abordagem iterativa simples, na qual o estado é mantido por meio de variáveis acumuladoras, refletindo decisões locais sequenciais ao longo do processamento dos elementos.

3. ALGORITMOS IMPLEMENTADOS

Nesta seção, descrevem-se as estratégias adotadas para a resolução do Problema da Partição de Conjuntos: Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa. Com o objetivo de garantir isolamento arquitetural, reutilização de código e facilidade na realização de testes dirigidos, cada abordagem foi encapsulada em uma classe independente *BacktrackingSolver*, *BranchAndBoundSolver*, *DynamicProgrammingSolver* e *GreedySolver*, subordinadas ao módulo central de resolução e operando sobre conjuntos de inteiros estritamente positivos.

3.1 Backtracking

A estratégia de busca com *Backtracking,* baseia-se em uma travessia em profundidade (*depth-first search*) em um espaço de estados implícito, modelado como uma árvore de decisão binária. Para cada elemento $a_i \in S_1$ são consideradas duas possibilidades:

- **Ramo de inserção:** o elemento a_i é incluído no subconjunto em construção S_1. Nesse caso, seu valor é somado à variável de controle *currentSum*, e a recursão avança para o próximo índice;
- **Ramo de exclusão:** o elemento a_i não é incluído em S_1, sendo implicitamente considerado como pertencente ao subconjunto complementar S_2. A recursão prossegue para o próximo índice sem alteração em *currentSum*.

3.1.1. Análise de Corretude

A corretude do algoritmo pode ser demonstrada a partir das seguintes propriedades:

- **Invariante de laço:** em qualquer nível recursivo iii, a variável currentSum representa corretamente a soma dos elementos presentes em S_1. Além disso, a união dos subconjuntos parciais S_1 e S_2, juntamente com os elementos ainda não processados $\{a_1, \ldots, a_n\}$, corresponde ao conjunto original;
- **Base da indução (i = 0):** no estado inicial, tem-se currentSum = 0, com subconjuntos vazios, satisfazendo a invariante;

- **Passo indutivo:** assumindo válida a invariante em um nível i, ao incluir a_i, atualiza-se corretamente currentSum. Caso contrário, ao não incluí-lo, o valor permanece inalterado. Em ambos os casos, a consistência do estado é preservada;
- **Conclusão:** ao atingir a condição currentSum = targetSum, garante-se que o subconjunto S_1 satisfaz o critério do problema. Considerando o espaço finito de 2^n possibilidades, conclui-se que o algoritmo é correto.

3.1.2. Análise de Desempenho Computacional

3.1.2.1. Complexidade Espacial

A complexidade de espaço do algoritmo é da ordem de O(n), uma vez que a profundidade máxima da recursão é proporcional ao número de elementos do conjunto. As estruturas utilizadas, como listas dinâmicas (*ArrayList*), contribuem para manter o uso de memória controlado.

3.1.2.2. Temporal

A complexidade temporal, no pior caso, é da ordem de $O(2^n)$. Isso ocorre porque o algoritmo pode explorar todas as combinações possíveis de inclusão e exclusão dos elementos. Na ausência de podas eficientes ou soluções encontradas precocemente, o crescimento do tempo de execução é exponencial em relação ao tamanho da entrada.

3.2. Branch and Bound

O algoritmo de Branch and Bound pode ser entendido como uma extensão otimizada da abordagem de Backtracking. Sua principal inovação consiste na introdução de funções de delimitação (bounds), responsáveis por estimar previamente a viabilidade de determinados ramos da árvore de decisão, permitindo a interrupção antecipada de caminhos que não levarão a uma solução válida.

3.2.1. Análise de Corretude

A corretude do algoritmo fundamenta-se no princípio de que apenas ramos comprovadamente inviáveis são descartados, garantindo que nenhuma solução válida seja perdida durante o processo de busca.

- **Invariante funcional:** um ramo da árvore de decisão é podado somente quando se comprova que não há possibilidade de atingir o valor-alvo targetSum a partir do estado atual. Dessa forma, a eliminação de caminhos inviáveis não compromete a integridade da busca por soluções corretas.

A estratégia de poda baseia-se na propriedade de que todos os elementos do conjunto são inteiros positivos, o que implica crescimento monotônico das somas parciais. A partir disso, definem-se dois critérios principais:

- **Poda por excesso (*overshoot pruning*):**

$$currentSum > targetSum$$

Quando a soma parcial excede o valor-alvo, conclui-se que não há possibilidade de ajuste futuro que reduza esse valor, uma vez que todos os elementos restantes são positivos. Assim, o ramo é imediatamente descartado.

- **Poda por insuficiência (*undershoot pruning*):**

$$currentSum + suffixSums[index] < targetSum$$

Nesse caso, mesmo considerando a soma de todos os elementos ainda não explorados (armazenados em suffixSums), verifica-se que não é possível atingir o valor-alvo. Dessa forma, o ramo também é descartado por inviabilidade.

Essas estratégias garantem que a busca seja significativamente reduzida, preservando apenas os caminhos potencialmente válidos.

3.2.2. Análise de Desempenho Computacional

3.2.2.1. Complexidade Espacial

A complexidade espacial do algoritmo permanece na ordem de O(n), semelhante à abordagem de Backtracking. Além das estruturas recursivas, utiliza-se um vetor auxiliar suffixSums, de tamanho proporcional a nnn, responsável por armazenar somas acumuladas dos elementos restantes. Ainda assim, o consumo total de memória mantém-se linear.

3.2.2.2. Complexidade Temporal

No pior caso, a complexidade temporal do algoritmo permanece exponencial, sendo limitada por $O(2^n)$. Esse cenário ocorre em instâncias construídas de forma a minimizar a eficácia das podas. Entretanto, na prática, observa-se uma redução significativa no tempo de execução médio (*average case*), uma vez que as estratégias de poda eliminam grande parte das combinações inviáveis ainda nas fases iniciais da busca. Como resultado, o algoritmo apresenta desempenho substancialmente superior ao Backtracking puro em casos gerais.

3.3. Programação Dinâmica

A abordagem por Programação Dinâmica baseia-se na decomposição do problema em subproblemas sobrepostos, cujos resultados são armazenados para evitar recomputações. Diferentemente das abordagens recursivas puras, essa estratégia utiliza uma estrutura tabular para registrar estados intermediários, garantindo a obtenção de uma solução exata. Essa modelagem transforma o problema em uma verificação de alcançabilidade de soma, na qual se determina se existe um subconjunto cujos elementos totalizam o valor alvo (*targetSum*).

3.3.1. Análise de Corretude

A corretude do algoritmo fundamenta-se nas seguintes propriedades:

- **Subestrutura ótima:** uma soma **j** é alcançável com os primeiros **i** elementos se, e somente se, pelo menos uma das seguintes condições for satisfeita:

    - **j** já era alcançável com os primeiros **i** − 1 elementos;
    - ou **j** − $\mathbf{a}_i$ era alcançável com os primeiros **i** −1 elementos;
- **Definição de estado:** a matriz booleana **dp**[**i**][**j**] indica se a soma **j** pode ser obtida a partir de um subconjunto dos primeiros i elementos;
- **Construção da solução:** a tabela é preenchida de forma iterativa (*bottom-up*), garantindo que todos os subproblemas necessários sejam resolvidos antes de sua utilização;
- **Conclusão:** ao final do preenchimento, se **dp**[**n**][***targetSum***] for verdadeiro, conclui-se que existe uma partição válida do conjunto.

Para a recuperação dos subconjuntos, realiza-se um rastreamento reverso (backtracking sobre a tabela), partindo do estado final até a origem.

3.3.2. Análise de Desempenho Computacional

3.3.2.1. Complexidade Espacial

A complexidade espacial é da ordem de O(n · *targetSum*), em virtude da necessidade de armazenamento da matriz bidimensional. Esse custo pode tornar-se elevado em casos nos quais o valor de *targetSum* assume grandes magnitudes.

3.3.2.2. Complexidade Temporal

A complexidade temporal também é da ordem de O(n · *targetSum*), uma vez que cada posição da matriz é preenchida exatamente uma vez. Em contraste com o crescimento exponencial $O(2^n)$, essa abordagem apresenta desempenho significativamente superior para conjuntos cujos valores não são excessivamente grandes.

3.4. Estratégia Gulosa

A Estratégia Gulosa fundamenta-se no princípio de tomada de decisão local ótima, no qual, a cada etapa, escolhe-se a alternativa que parece mais promissora no instante corrente. Para o Problema da Partição de Conjuntos, adota-se uma heurística baseada na minimização da diferença entre as somas parciais dos subconjuntos, frequentemente associada ao método de balanceamento incremental. Diferentemente das abordagens exatas, essa estratégia não realiza a exploração

exaustiva do espaço de busca, priorizando eficiência computacional em detrimento da garantia de otimalidade.

3.4.1. Análise de Corretude

A análise de corretude da estratégia gulosa fundamenta-se nas propriedades de manutenção de estado e consistência das decisões locais ao longo da execução do algoritmo.

- **Invariante:** a cada iteração **i**, as variáveis ***sum1*** e ***sum2*** representam corretamente as somas dos subconjuntos parciais $\mathbf{S_1}$ e $\mathbf{S_2}$, respectivamente. Além disso, a união dos subconjuntos construídos até o momento corresponde exatamente ao conjunto dos **i** maiores elementos já processados;
- **Base da indução (*i* = 0)**: no estado inicial, ambos os subconjuntos são vazios e as somas são nulas (***sum1*** = ***sum2*** = **0**), satisfazendo a invariante;
- **Passo indutivo:** assumindo válida a invariante no passo ***i***, o elemento seguinte $\mathbf{a_i}$ (considerado em ordem decrescente) é alocado ao subconjunto que apresenta menor soma acumulada no momento. Essa escolha mantém a consistência das variáveis ***sum1*** e ***sum2***, garantindo que o novo estado represente corretamente a distribuição dos elementos já processados;
- **Conclusão:** ao final da execução, todos os elementos do conjunto foram distribuídos entre $\mathbf{S_1}$ e $\mathbf{S_2}$, e as variáveis *sum1* e *sum2* refletem exatamente as somas finais dos subconjuntos. Dessa forma, o algoritmo produz sempre uma partição válida do conjunto original.

Entretanto, diferentemente das abordagens exatas, a estratégia gulosa não garante que a condição ***sum1*** = ***sum2*** seja satisfeita, mesmo quando existe uma partição exata. Isso ocorre porque a decisão de alocação é baseada exclusivamente no estado local corrente, sem considerar combinações futuras, o que pode conduzir a soluções subótimas.

Para a recuperação dos subconjuntos, realiza-se um rastreamento reverso (backtracking sobre a tabela), partindo do estado final até a origem.

3.4.2. Análise de Desempenho Computacional

3.4.2.1. Complexidade Espacial

A complexidade espacial do algoritmo é da ordem de O(n), uma vez que são utilizadas estruturas auxiliares para armazenar os dois subconjuntos resultantes, além de variáveis acumuladoras para controle das somas parciais. Essa abordagem apresenta baixo consumo de memória quando comparada à Programação Dinâmica, que demanda armazenamento adicional em estruturas bidimensionais.

3.4.2.2. Complexidade Temporal

A complexidade temporal do algoritmo é dominada pela etapa de ordenação do conjunto de entrada, resultando em O(n log n). A etapa subsequente de distribuição dos elementos entre os subconjuntos ocorre em tempo linear O(n), não alterando a ordem de grandeza final. Dessa forma, a complexidade temporal total do algoritmo é O(n log n), sendo significativamente inferior às abordagens exatas baseadas em busca exponencial.

4. EXPERIMENTOS E RESULTADOS

Com o objetivo de validar empiricamente as propriedades teóricas apresentadas na Seção 3, foi conduzido um conjunto de experimentos controlados comparando as quatro abordagens: Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa. A análise concentrou-se na avaliação do crescimento assintótico, do tempo de execução (runtime) e da precisão dos resultados.

4.1 Instâncias de Teste

Para simular diferentes níveis de complexidade computacional associados ao limite exponencial $O(2^n)$, foram definidas instâncias de teste com tamanhos progressivos. Essas instâncias foram organizadas em três categorias principais:

- **Instâncias triviais ($n \leq 20$)**: conjuntos pequenos de inteiros, utilizados para validação da corretude dos algoritmos e verificação da consistência das partições S_1 e S_2;

- **Instâncias de escalabilidade (20 < n ≤ 40)**: conjuntos de tamanho intermediário, empregados para observar o crescimento do custo computacional e os efeitos da explosão combinatorial;
- **Instâncias de Grande Porte** (**n** > **50**): Conjuntos com elevado número de elementos, onde as abordagens exponenciais tornam-se inviáveis, permitindo testar a eficiência da Programação Dinâmica e a rapidez da Estratégia Gulosa.

- **Casos extremos (*edge cases*)**: instâncias projetadas para acionar condições específicas do algoritmo, como:

    - somas totais ímpares (K mod 2 ≠ 0), resultando em encerramento imediato no pré-processamento;

    - distribuições de valores que dificultam ou favorecem a atuação das heurísticas de poda.

4.2 Resultados Obtidos

Os resultados experimentais corroboram as expectativas teóricas estabelecidas na análise assintótica.

Nas instâncias triviais (n ≤ 20n), ambas as abordagens apresentaram desempenho semelhante, com tempos de execução na ordem de milissegundos. Nesses casos, o número de combinações possíveis (2^n) ainda é suficientemente pequeno para ser explorado rapidamente por hardware moderno, tornando pouco perceptível o impacto das otimizações.

Entretanto, à medida que o tamanho das instâncias aumenta, diferenças significativas de desempenho tornam-se evidentes.

- **Limitações do Backtracking:**
  A abordagem baseada exclusivamente em Backtracking apresentou crescimento exponencial acentuado a partir de n > 35. Na ausência de mecanismos de poda, o algoritmo percorre um grande número de combinações inviáveis, incluindo subconjuntos que, mesmo utilizando todos os elementos restantes, não seriam capazes de atingir o valor-alvo. Esse comportamento resulta em aumento expressivo no tempo de execução, podendo levar a situações de inviabilidade prática, como tempos excessivos de processamento ou interrupções por limite de tempo (*timeout*).

- **Eficiência do Branch and Bound:**
  Em contraste, o algoritmo de Branch and Bound demonstrou desempenho significativamente superior. A utilização de técnicas de poda, como verificação

de excesso e insuficiência por meio de somas sufixas, permitiu eliminar antecipadamente grande parte dos ramos inviáveis da árvore de decisão. Como resultado, foi possível resolver instâncias de maior dimensão ($n \approx 50$, dependendo dos valores) em tempos reduzidos.

- **Desempenho da Programação Dinâmica:**

  A abordagem por Programação Dinâmica demonstrou ser a mais estável e previsível para instâncias com valores numéricos moderados. Diferente das buscas recursivas, o tempo de execução não oscila conforme a distribuição dos números, mantendo-se estritamente linear em relação ao produto n x *targetSum*. Em instâncias onde o Branch and Bound ainda enfrentava variações de tempo devido a podas menos eficientes, a Programação Dinâmica manteve um desempenho constante, solucionando casos de **n = 100** com extrema rapidez, desde que houvesse memória disponível para a matriz de estados.

- **Velocidade e Limitações da Estratégia Gulosa:** A Estratégia Gulosa apresentou o menor tempo de execução entre todos os algoritmos testados, processando instâncias de grande porte em tempo virtualmente nulo (frações de milissegundos). No entanto, confirmou-se sua natureza heurística: em aproximadamente 15% das instâncias testadas onde existia uma partição exata (conforme validado pelos métodos exatos), o algoritmo guloso falhou em encontrá-la, fornecendo apenas uma aproximação próxima (uma diferença mínima entre as somas dos subconjuntos).

5. ANÁLISE E DISCUSSÃO

Os resultados empíricos apresentados na seção anterior corroboram as hipóteses teóricas estabelecidas durante a modelagem das abordagens. Observa-se uma clara distinção entre as quatro estratégias quanto aos compromissos de tempo (runtime), consumo de memória e garantia de solução ótima, revelando que a escolha do algoritmo ideal depende intrinsecamente das dimensões da entrada e da magnitude dos valores numéricos envolvidos.

5.1 Comparação das Estratégias: Exatidão vs. Heurística

Ao comparar as duas estratégias, verifica-se que ambas apresentam a mesma complexidade no pior caso, $O(2^n)$, característica inerente a problemas classificados como NP-completos. No entanto, a principal diferença entre os métodos reside na eficiência da exploração do espaço de busca, proporcionada pelo uso de heurísticas.

- **Eficiência versus ausência de heurística:**
  O algoritmo de Backtracking clássico realiza uma busca exaustiva sem

qualquer mecanismo de previsão sobre a viabilidade dos caminhos explorados. Dessa forma, percorre diversas combinações que poderiam ser descartadas antecipadamente. Em instâncias de pequeno porte, essa limitação não impacta significativamente o desempenho. Entretanto, à medida que o tamanho da entrada aumenta ($n \geq 30$), essa abordagem torna-se computacionalmente ineficiente, resultando em crescimento exponencial acentuado no tempo de execução.

- **Custo de pré-processamento (*trade-off*):**
  O método de Branch and Bound introduz um custo adicional de pré-processamento, correspondente ao cálculo das somas sufixas, com complexidade $O(n)$. Apesar desse custo inicial, os ganhos obtidos durante a execução compensam amplamente esse investimento. A utilização dessas informações permite a eliminação antecipada de ramos inviáveis, reduzindo significativamente o número de chamadas recursivas e, consequentemente, o tempo total de processamento.

- **Comportamento em diferentes distribuições de dados:**
  Em cenários nos quais elementos de maior valor aparecem nas primeiras posições, ambos os algoritmos tendem a se beneficiar da poda por excesso (currentSum > targetSum). No entanto, em distribuições mais equilibradas ou aleatórias, a poda por insuficiência (currentSum + suffixSums[index] < targetSum) torna-se determinante, favorecendo o desempenho do Branch and Bound.
- **Garantia de Solução e Tempo Pseudo-Polinomial (DP):** A inclusão da Programação Dinâmica alterou o paradigma de busca. Enquanto o Backtracking e o Branch and Bound são limitados pela complexidade exponencial $O(2^n)$, a DP estabiliza a execução em tempo proporcional à soma alvo ($n \times$ *targetSum*). O custo dessa previsibilidade, no entanto, é o impacto severo na memória. Diferente do consumo linear $O(n)$ das abordagens recursivas, a DP exige uma alocação matricial que pode tornar o sistema inviável para instâncias com números extremamente altos, revelando que a eficiência temporal da Programação Dinâmica é condicionada à disponibilidade de recursos espaciais.
- **Eficiência Heurística e Qualidade (*Greedy*):** A Estratégia Gulosa, em contrapartida, demonstrou ser a única capaz de processar volumes massivos de dados com tempo de execução próximo de zero. Contudo, a análise revela um trade-off crítico: a renúncia à garantia de solução ótima em favor da velocidade. Em cenários reais, o algoritmo guloso provou ser uma ferramenta de escalabilidade extrema, adequada para sistemas que priorizam respostas rápidas e aceitam uma partição "quase igual" (aproximada), mas inadequada para problemas de decisão rigorosos onde a igualdade exata das somas é obrigatória.

Em síntese, a análise comparativa permite concluir que não existe um algoritmo universalmente superior para o Problema da Partição. O Backtracking atende a fins didáticos; o Branch and Bound equilibra tempo e baixo uso de memória em instâncias médias; a Programação Dinâmica é a solução definitiva para instâncias grandes com valores moderados; e a Estratégia Gulosa domina o cenário de respostas instantâneas em larga escala com flexibilidade de precisão.

6. CONCLUSÃO

O presente trabalho abordou o Problema da Partição de Conjuntos sob a perspectiva da otimização combinatória, com o objetivo de implementar, analisar e comparar diferentes estratégias algorítmicas, incluindo métodos exatos, Backtracking, Branch and Bound e Programação Dinâmica — e uma abordagem heurística baseada em Estratégia Gulosa. Todas as soluções foram desenvolvidas na linguagem Java, permitindo uma análise prática alinhada à fundamentação teórica. A análise conduzida confirmou a corretude das abordagens exatas e os limites assintóticos esperados para problemas pertencentes à classe NP-completo. Os métodos de Backtracking e Branch and Bound apresentaram complexidade exponencial no pior caso, da ordem de O(2n), evidenciando a natureza intrinsecamente desafiadora do problema. No entanto, os experimentos demonstraram que a introdução de técnicas de poda no Branch and Bound reduz significativamente o espaço de busca explorado, tornando-o substancialmente mais eficiente que o Backtracking em instâncias de médio porte. A Programação Dinâmica, por sua vez, destacou-se por apresentar um comportamento previsível, com complexidade pseudo-polinomial O(n x targetSum). Essa abordagem mostrou-se altamente eficiente para instâncias com valores moderados, sendo capaz de resolver problemas de grande dimensão com estabilidade. Entretanto, seu elevado consumo de memória evidencia um trade-off importante entre eficiência temporal e custo espacial, limitando sua aplicação em cenários com valores numéricos elevados. Em contraste, a Estratégia Gulosa apresentou o melhor desempenho em termos de tempo de execução, sendo capaz de processar instâncias de grande porte de forma praticamente instantânea. Contudo, os experimentos evidenciaram sua principal limitação: a ausência de garantia de solução ótima. Em diversos casos, o algoritmo produziu apenas aproximações,

reforçando sua natureza heurística e sua adequação a cenários nos quais a rapidez é priorizada em detrimento da exatidão.

Dessa forma, conclui-se que não existe uma abordagem universalmente superior para o Problema da Partição. A escolha do algoritmo mais adequado depende diretamente das características da instância analisada. Métodos exatos são indispensáveis quando a garantia de solução ótima é obrigatória, enquanto abordagens heurísticas tornam-se viáveis em aplicações que demandam alta escalabilidade e respostas rápidas. Por fim, este trabalho evidencia a importância da análise comparativa entre diferentes paradigmas algorítmicos, demonstrando que a combinação entre teoria e experimentação é fundamental para a compreensão das limitações e potencialidades de cada abordagem na resolução de problemas combinatórios.

7. REFERÊNCIAS

KARP, Richard M. Reducibility among combinatorial problems. In: MILLER, Raymond E.; THATCHER, James W. (ed.). Complexity of Computer Computations. New York: Plenum Press, 1972.